



Verstehen und Einordnen von Ressourcen im K8s Cluster



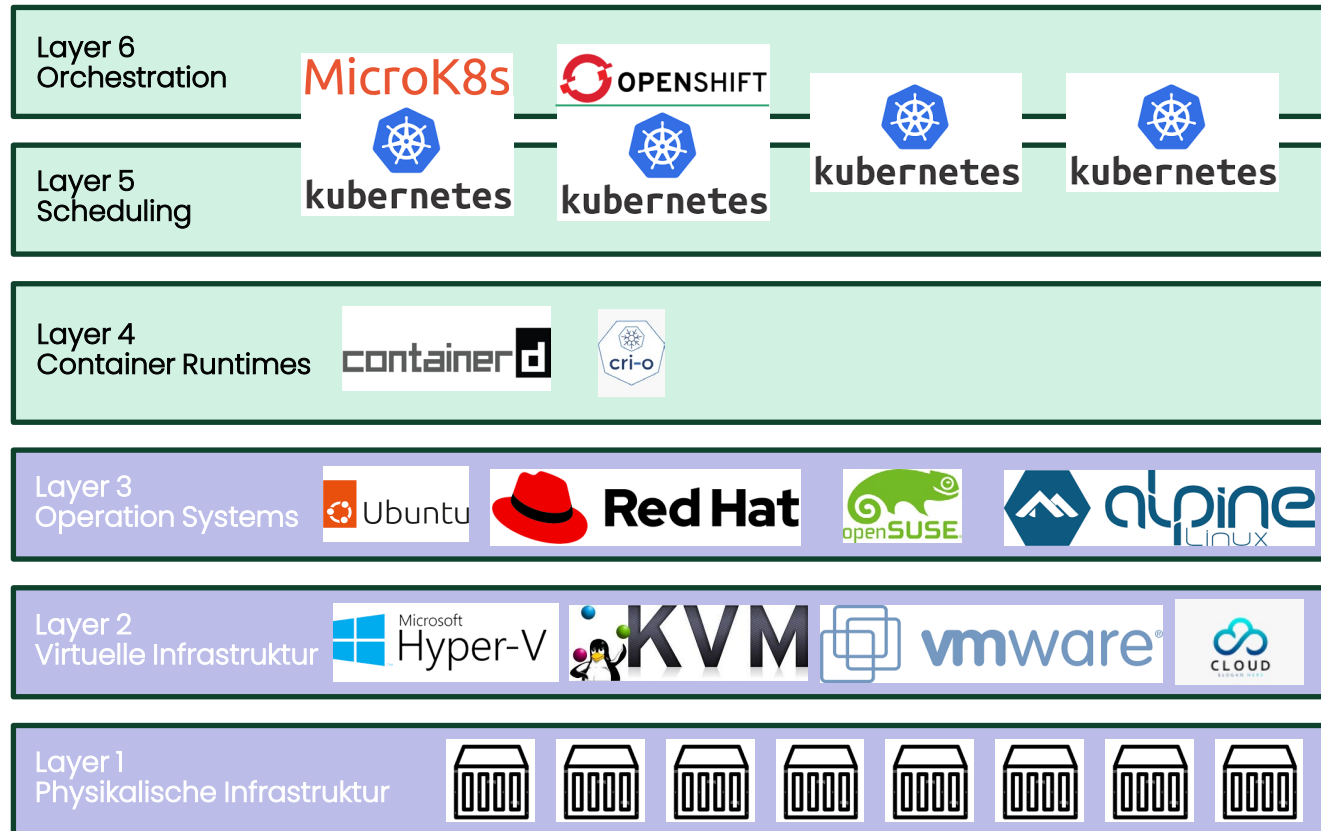
Lernziele

- Sie haben einen Überblick über die Ressourcen im K8s Cluster und können diese Einordnen.

Zeitlicher Ablauf

- Kubectl-CLI
- YAML (Dateien)
- K8s Ressourcen (Objects) jeweils mit 1 – 3 Übungen
 - ◇ Basis Ressourcen (Pod, Labels, Selector, Service)
 - ◇ Netzwerk Ressource (Ingress)
 - ◇ Ressourcen für Applikations-Verteilung und Updates (ReplicaSet, Deployment)
 - ◇ Ressourcen für Persistenz (Volume, PersistentVolume, PersistentVolumeClaim, StorageClass)
 - ◇ Ressourcen für Konfiguration (ConfigMap, Secrets)
- Reflexion
- Lernzielkontrolle
- Bonus: Spezial Ressourcen (StatefullSets, Jobs, DaemonSet, Health Probe Pattern etc.)

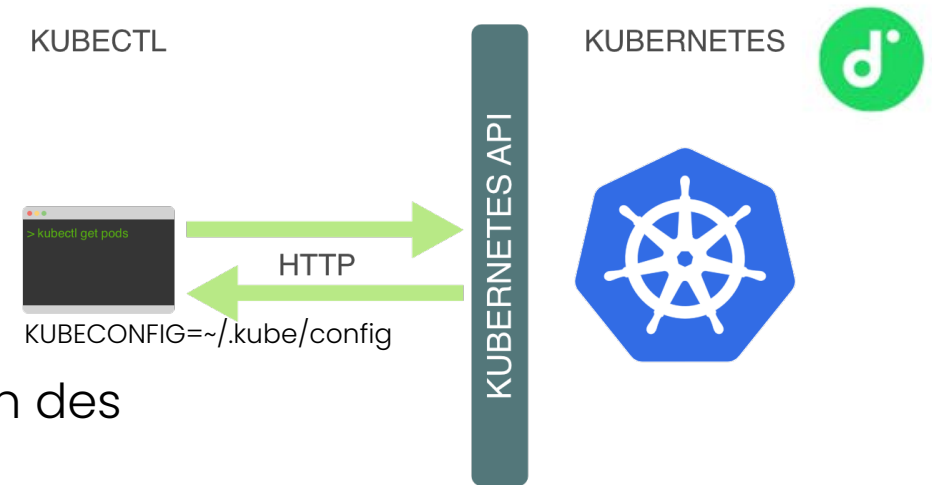
Die (vereinfachten) »Layer« der Container-Welt



- Nun geht es um die Layer 5 und 6

Die (vereinfachten) »Layer« der Container-Welt

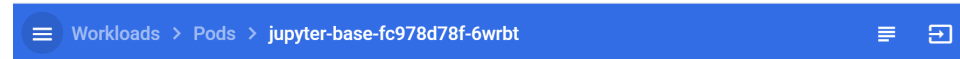
Kubectl-CLI (1)



- Das kubectl-Kommando stellt, eine der Schaltzentralen des K8s Clusters zur Administration der Ressourcen dar.
- Die wichtigsten kubectl-Subkommandos in der Übersicht
 - ◇ **Container Images von Docker nach Kubernetes (Imperative)**
 - `kubectl run name --image=image` – startet ein Container (ähnlich docker run)
 - `kubectl expose` – öffnet Port gegen aussen
 - `kubectl create name --image=image` – Erstellt eine Ressource
 - ◇ **Deklarativ**
 - `kubectl apply -f YAML` – führt die Änderungen in der YAML im Cluster nach
 - `kubectl delete -f YAML` – Löscht eine Ressource laut YAML Datei/Verzeichnis
 - `kubectl get [all] [-o yaml]` – zeigt Ressourcen, in unterschiedlichen Formaten, an

Kubectl-CLI (2)

- **Informationen zu den Ressourcen**
 - ◇ `kubectl describe type/resource` – zeigt Details einer Ressource an
 - ◇ `kubectl logs pod` – zeige das Log eines Containers ([/dev/stdout](#), [/dev/stderr](#))
- **Fehlerbehebung (Troubleshooting, Debugging)**
 - ◇ `kubectl cluster-info dump` – Liefert Detailinformationen zum K8s Cluster
 - ◇ `kubectl exec` – startet einen Befehl im Container oder wechselt in Container (entspricht `sudo nsenter -t <pid> -a sh`)
 - ◇ `kubectl debug` – (ab 1.25) Debuggen mittels flüchtiger Container ([ephemeral containers](#))
- **Portweiterleitung (port forwarding)**
 - ◇ `kubectl port-forward` – Port eines Service innerhalb des Clusters an lokales System weiterleiten (entspricht `ssh -L <port>:localhost:<port> <ip Kubernetes VM>.`)
 - ◇ `kubectl proxy` – Aufrufe an den API-Service ab lokalem System ermöglichen (wie port-forward – Kurzform für Kubernetes API).
- [kubectl Cheatsheet](#) und [9 kubectl commands sysadmins need to know](#)





Welche Ressourcen/Aktionen können im K8s Cluster via kubectl-CLI verwaltet werden?

NAME	SHORTNAMES	APIVERSION	NAMESPACED
configmaps	cm	v1	true
endpoints	ep	v1	true
events	ev	v1	true
limitranges	limits	v1	true
namespaces	ns	v1	false
nodes	no	v1	false
persistentvolumeclaims	pvc	v1	true
persistentvolumes	pv	v1	false
Pods	po	v1	true
replicationcontrollers	rc	v1	true
resourcequotas	quota	v1	true
serviceaccounts	sa	v1	true
services	svc	v1	true
daemonsets	ds	apps/v1	true
deployments	deploy	apps/v1	true
replicasets	rs	apps/v1	true
statefulsets	sts	apps/v1	true
cronjobs	cj	batch/v1	true
jobs		batch/v1	true

YAML (Dateien)

```
# Order
apiVersion: v1
kind: Service
metadata:
  name: order-standalone
  labels:
    app: scsesi-order-standalone
    group: ewolff-scsesi
    tier: frontend
spec:
  type: NodePort
  ports:
    - port: 8080
      nodePort: 32090
  selector:
    app: scsesi-order-standalone
```

Diagramm zur YAML-Struktur:

- [Gruppe /] Version: `apiVersion: v1`
- K8s Ressource (Object): `kind: Service`
- Abschnitt (Einrückung!): `metadata:` (enthält `name` und `labels`)
- '-' = Array: `ports:` (enthält eine Liste von Ports)
- Key: Value: `selector:` (enthält `app: scsesi-order-standalone`)

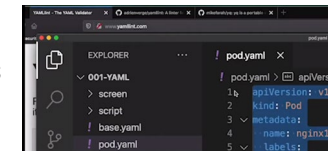
Achtung

Wichtig bei allen folgenden Yaml/JSON-Files sind natürlich insbesondere die korrekten Einrückungen, da über sie die Hierarchien der Direktiven definiert werden.

Zudem ist darauf zu achten, dass die Verwendung von Tabs bzw. die Vermischung von Tabs und Spaces problematisch sein kann. Insofern sollten für alle Einrückungen am besten durchgängig Spaces verwendet werden.

- [YAML](#) ist eine vereinfachte Auszeichnungs-sprache (englisch markup language) zur Datenserialisierung, angelehnt an XML (ursprünglich) und an die Datenstrukturen in den Sprachen Perl, Python und C sowie dem in [RFC2822](#) vorgestellten E-Mail-Format.
- Die grundsätzliche Annahme von YAML ist, dass sich jede beliebige Datenstruktur nur mit assoziativen Listen, Listen (Arrays) und Einzelwerten (Skalaren) darstellen lässt. Durch dieses einfache Konzept ist YAML wesentlich leichter von Menschen zu lesen und zu schreiben als beispielsweise XML, ausserdem vereinfacht es die Weiterverarbeitung der Daten, da die meisten Sprachen solche Konstrukte bereits integriert haben.

- [YAML tips](#) for Kubernetes



Inhalt YAML Datei (API + Ressource)
apiVersion: <API-Gruppe>/<Version>
kind: <Ressource>



API-Gruppen / Ressource

- Die Ressourcen in Kubernetes gliedern sich in **API-Gruppen**.
- Jede **API-Gruppe** ist für einen bestimmten Zweck zuständig, z.B. Core, Network, Storage etc.
- Fehlt die API-Gruppe handelt es sich um eine Kubernetes Core Ressource.

Der **Erweiterungsmechanismus** von Kubernetes ermöglicht es, die Plattform mittels neuer API-Gruppen durch zusätzliche Funktionen und Ressourcen zu erweitern.

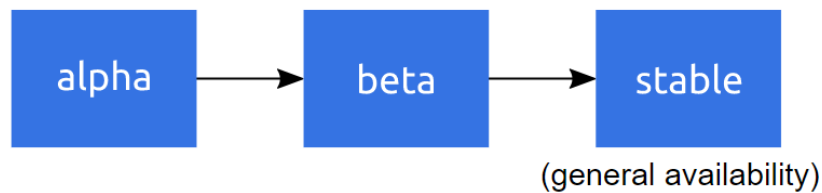
- Dazu werden sogenannte **Custom Resource Definitions (CRDs)** verwendet.
- **Custom Resource Definitions (CRDs):**
 - ◇ CRDs sind benutzerdefinierte Ressourcen, die von Entwicklern erstellt werden können. Sie ermöglichen die Definition neuer Ressource typen (Objects), die über die Standard-Kubernetes-Ressourcen hinausgehen.
 - ◇ Beispielsweise kann ein CRD für eine Anwendung definiert werden, die spezifische Eigenschaften und Verhaltensweisen hat, die nicht von den Standard-Ressourcen von Kubernetes abgedeckt werden.
- Der Eintrag **kind:** repräsentiert die eigentliche Ressource.

Inhalt YAML Datei (API + Ressource)
apiVersion: <API-Gruppe>/<Version>
kind: <Ressource>



API-Versionierung

- Nicht jedes der vorgenannten Objekte bzw. jede Ressource kann im K8s Cluster »einfach mal so« ausgerollt werden.
- Entscheidend darüber, ob unser Kubernetes Cluster unsere Eingabe akzeptiert, ist das Attribut apiVersion in der YAML Datei, welches im Default auf v1 (Version 1) eingestellt ist.



Alpha

- eventuell buggy
- standardmässig deaktiviert
- Support kann in zukünftigen Versionen wieder gestrichen werden
- API kann sich ändern
- nur für Testumgebungen

Beta

- gut getesteter und (**nach den Massstäben der Container-Welt**) als sicher zu betrachtender Code
- Support für Features bleibt bestehen
- Kleine Details können sich ändern
- Nicht für »Business Critical«-Anwendungen gedacht

Stable: Stabil



Übung 07-1: api-resources

kubectl api-resources

Der Befehl `kubectl api-resources` ist ein nützliches Werkzeug innerhalb der Kubernetes-Befehlszeilenschnittstelle (CLI), das Administratoren und Entwicklern hilft, die verschiedenen verfügbaren API-Ressourcen in einem Kubernetes-Cluster zu identifizieren.

Mit `kubectl api-resources` können Benutzer eine Liste aller Ressourcenarten abrufen, die im aktuellen Cluster unterstützt werden, einschliesslich ihrer Namen, Kurzformen, API-Gruppen und ob sie namespacespezifisch sind oder nicht.

Dies erleichtert die Navigation und Verwaltung der komplexen Kubernetes-API und unterstützt bei der effizienten Steuerung und Überwachung der Clusterressourcen.

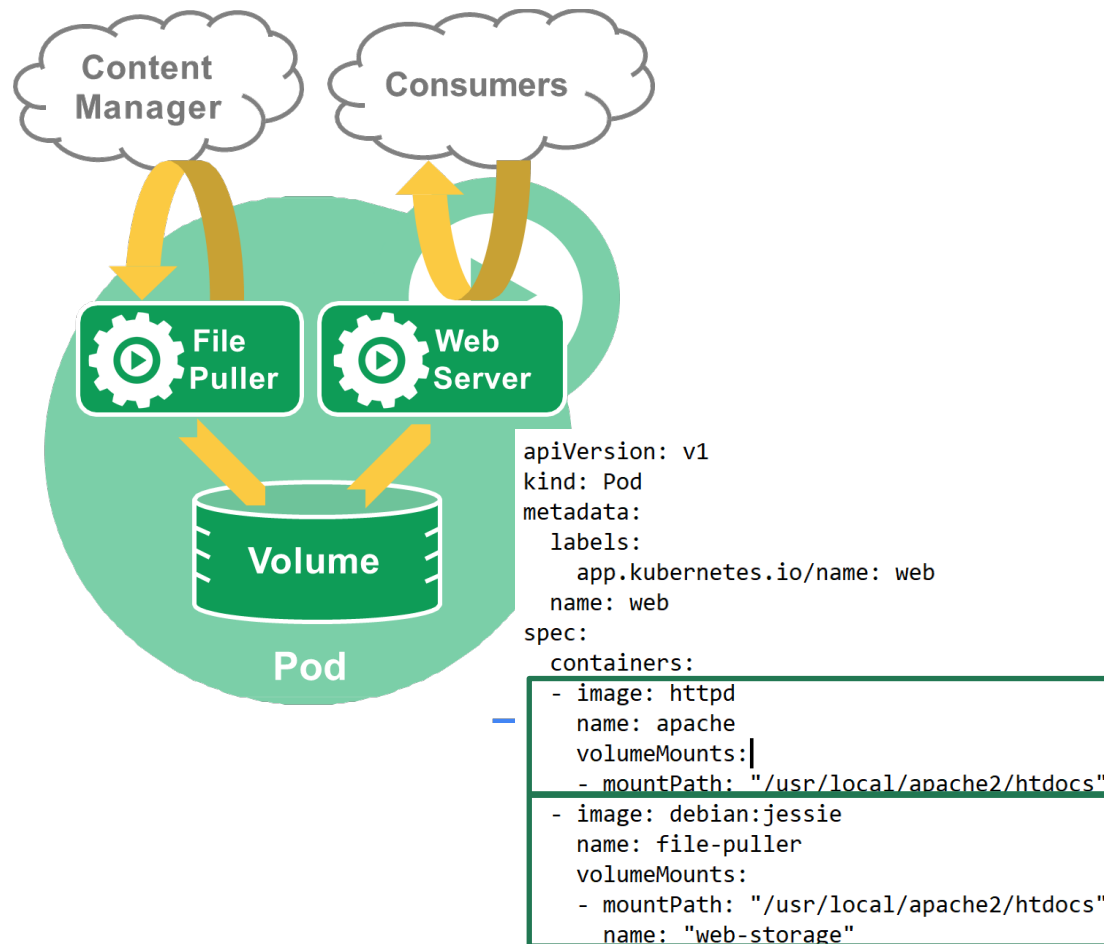
Zuerst schauen wir uns alle Kubernetes Ressourcen an:

```
In [ ]: ! kubectl api-resources
```

Die einzelnen Felder haben folgende Bedeutung:

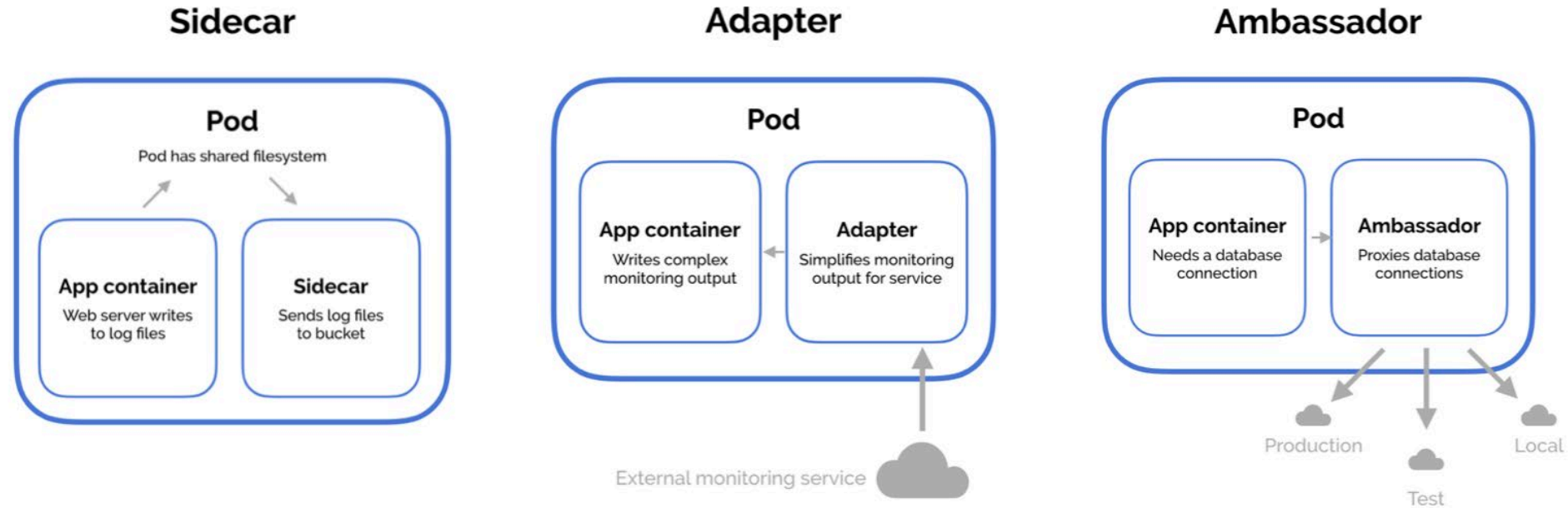
- **NAME:** Diese Spalte gibt den vollständigen Namen der API-Ressource an. Beispiele hierfür sind "pods", "services" oder "deployments". Der Name ist der Hauptidentifikator für die Ressource innerhalb von Kubernetes.
- **SHORTNAMES:** Dies sind die Kurzformen oder Abkürzungen, die für die API-Ressourcen verwendet werden können, um die Eingabe in der Befehlszeile zu verkürzen. Zum Beispiel kann "pods" auch mit "po" abgekürzt werden. Diese Kurzformen erleichtern die Arbeit mit `kubectl`.
- **APIVERSION:** Diese Spalte zeigt die API-Version, unter der die Ressource definiert ist. Die API-Version besteht aus einer Gruppennamen und einer Versionsnummer, z. B. "v1" oder "apps/v1". Die API-Version gibt an, welche Version des Kubernetes-API-Standards verwendet wird und zu welcher API-Gruppe die Ressource gehört.
- **NAMESPACED:** Diese Spalte zeigt an, ob die Ressource innerhalb eines Namespaces existiert oder nicht. Ressourcen, die als "true" markiert sind, sind namespacespezifisch, das heißt, sie existieren innerhalb eines bestimmten Namespaces. Ressourcen, die als "false" markiert sind, existieren clusterweit und sind nicht an einen bestimmten Namespace gebunden.
- **KIND:** Diese Spalte gibt den Typ der Ressource an, wie er im Kubernetes-API-Objektmodell definiert ist. Beispiele hierfür sind "Pod", "Service", "Deployment" oder "ConfigMap". Der Typ beschreibt die Art der Ressource und definiert ihre Struktur und ihr Verhalten im Cluster.

Pods



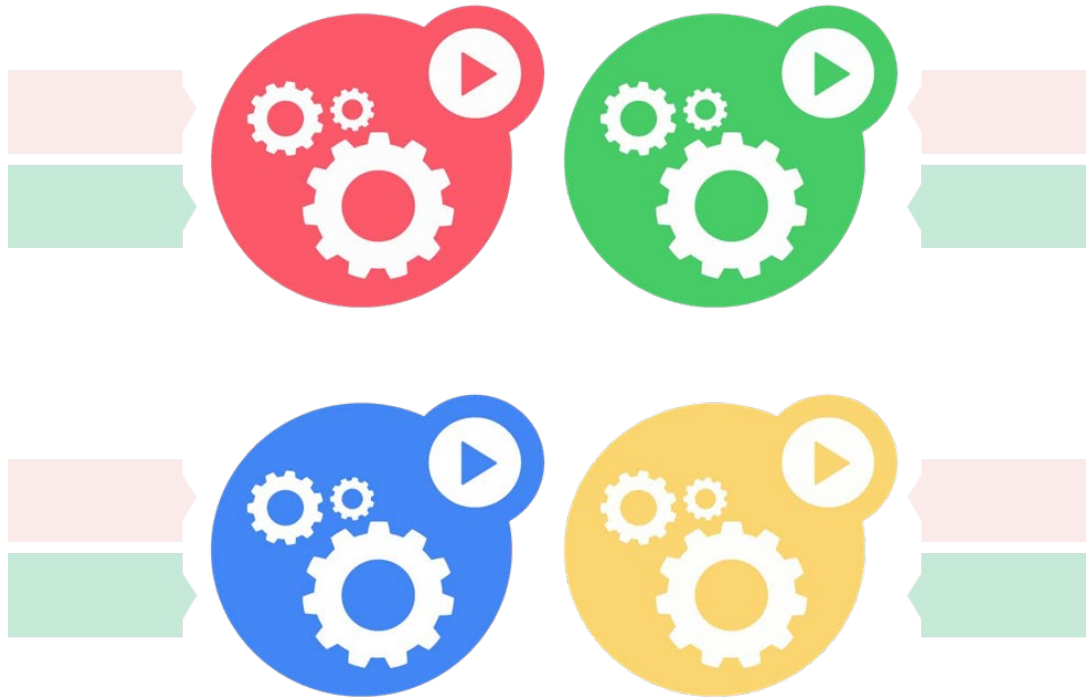
- Kleine Gruppe von Containern welche eng verbunden sind.
 - ◊ Kleinste Einheit für Replikation und Platzierung (auf Node).
 - ◊ "Logischer" Host für Container
- Jeder Pods erhält genau eine IP-Adresse
 - ◊ share data: localhost, volumes, IPC, etc.
- Erleichtert zusammengesetzte Applikationen
- Beispiele: file puller (holt Daten, schreibt index.html), web server (zeigt Daten an).

Architekturmuster: Multicontainer Pods



- Sidecar: eigentlicher Microservices, Sidecar z.B. für die Verschlüsselung der Kommunikation (Istio)
- Adapter: Microservice und Adapter welcher Protokoll A (MQTT) nach B (HTTP) wandelt.
- Ambassador: Microservices Verbindung DB = localhost, Ambassador Weiterleitung an effektive DB

Labels



- Vom User bereitgestellte **Schlüssel** und **Werte** (key=value)
- Kennzeichnung eines API-Objekt
labels:

```
app: my-app  
tier: FE
```
- Abfragbar mittels Selektoren (selectors), ähnlich SQL

```
kubectl get pods -l tier=FE
```
- Der **einzige** Gruppierungsmechanismus

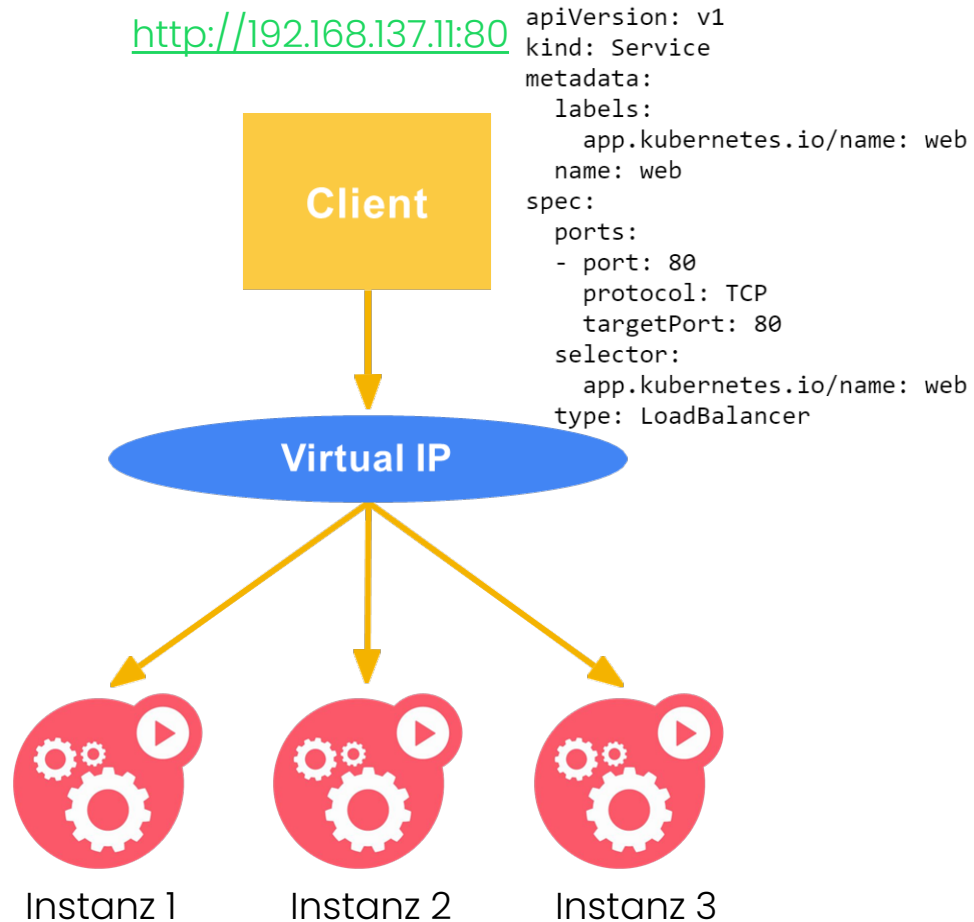
Recommended Labels (Empfohlene Labels)

Key	Description	Example	Type
<code>app.kubernetes.io/name</code>	The name of the application	<code>mysql</code>	string
<code>app.kubernetes.io/instance</code>	A unique name identifying the instance of an application	<code>wordpress-abcxyz</code>	string
<code>app.kubernetes.io/version</code>	The current version of the application (e.g., a semantic version, revision hash, etc.)	<code>5.7.21</code>	string
<code>app.kubernetes.io/component</code>	The component within the architecture	<code>database</code>	string
<code>app.kubernetes.io/part-of</code>	The name of a higher level application this one is part of	<code>wordpress</code>	string
<code>app.kubernetes.io/managed-by</code>	The tool being used to manage the operation of an application	<code>helm</code>	string

To illustrate these labels in action, consider the following StatefulSet object:

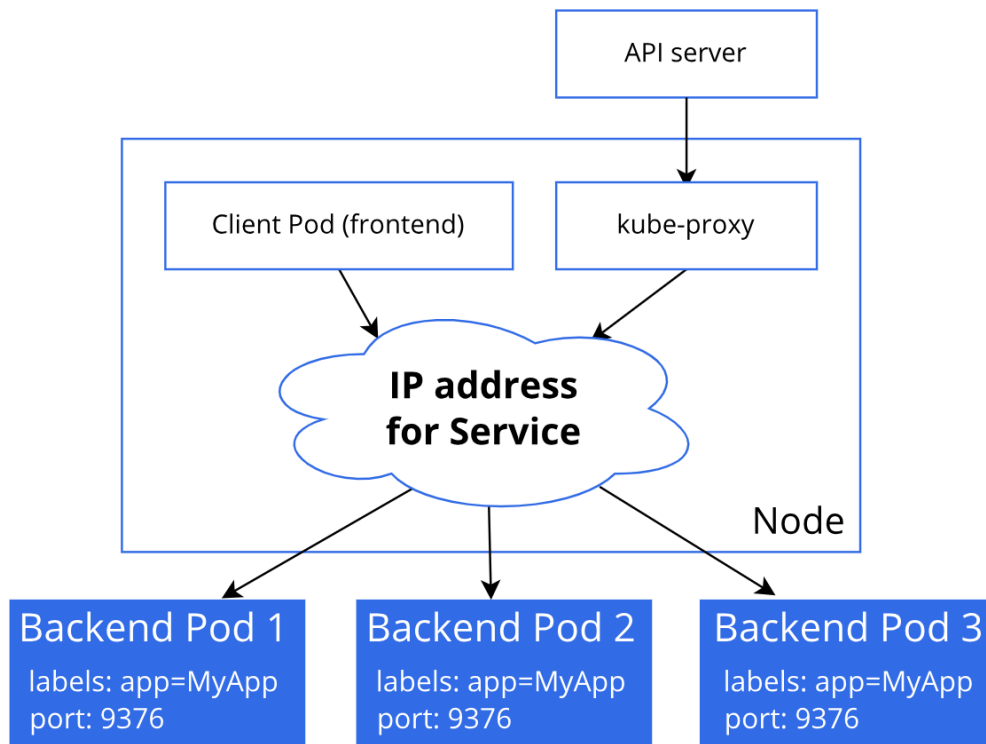
```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  labels:
    app.kubernetes.io/name: mysql
    app.kubernetes.io/instance: wordpress-abcxyz
    app.kubernetes.io/version: "5.7.21"
    app.kubernetes.io/component: database
    app.kubernetes.io/part-of: wordpress
    app.kubernetes.io/managed-by: helm
```

Services (port-based routing)



- Eine Gruppe von Pods die zusammenarbeiten, Gruppirt mittels Label Selector (i.d.R. mehrere Instanzen eines Microservices)
- Erlaubt mittels unterschiedlicher Methoden auf den Service zuzugreifen:
 - ◊ DNS Name und [DNS SRV](#) (Service Resource Record)
 - ◊ [Kubernetes Endpoints](#) API
- Definiert die Sicherbarkeit des Services ([ServiceTypes](#)):
 - ◊ ClusterIP: nur innerhalb des Clusters erreichbar (default).
 - ◊ NodePort: via Port und IP Worker Node erreichbar.
 - ◊ LoadBalancer: mittels eigener IP Adresse erreichbar (dazu muss ein LoadBalancer vorhanden sein). Ansonsten verhalten wie NodePort.
 - ◊ ExternalName: Der Service leitet einfach DNS-Anfragen auf **einen externen Hostnamen** weiter. Kein ClusterIP, keine Proxy-Verbindung.
- Versteckt Komplexität.
- **Beispiel** (Web Server)
 - ◊ ClusterIP: <http://web:80>
 - ◊ NodePort: <http://<Worker-Node-IP>:3xxx>
 - ◊ LoadBalancer: z.B. <http://192.168.137.11:80>

Services (K8s Dienst Kube-Proxy)



- Der K8s-Netzwerk-Proxy – er arbeitet auf jedem Worker Node und kümmert sich um die eingehenden Requests und ihr Routing.
- Er kann als primitiver Loadbalancer fungieren

```

r-01:~$ kubectl get pods --all-namespaces
NAME                                READY
coredns-5644d7b6d9-dsxh7           1/1
coredns-5644d7b6d9-ndwjq           1/1
etcd-master-01                      1/1
kube-apiserver-master-01            1/1
kube-controller-manager-master-01  1/1
kube-flannel-ds-amd64-pn94s         1/1
kube-proxy-j7fb2                    1/1
kube-scheduler-master-01            1/1
  
```

- Weitere Informationen:
 - ◇ [How To Inspect Kubernetes Networking](#)
 - ◇ Kubernetes [NodePort and iptables rule](#)
 - ◇ Install a Kubernetes [load balancer](#)
 - ◇ (microk8s enable metallb 192.168.50.0/24)



Übung 09-1: Basis Ressourcen

Übung: kubectl-CLI und Basis Ressourcen

Das `kubectl` -Kommando stellt, eine der Schaltzentralen des K8s Clusters zur Administration der Ressourcen dar.

In dieser Übung verwenden wir das `kubectl` -Kommando zur Erstellen eines Pod's und Services nutzen.

Das passiert in einer eigenen Namespace um die Resultate gezielt Darstellen zu können:

```
! kubectl create namespace test
```

```
namespace/test created
```

Erzeugen eines Pod's, hier der Apache Web Server.

Die Option `--restart=Never` erzeugt nur einen Pod. Ansonsten wird ein Deployment erzeugt.

```
! kubectl run apache --image=httpd --restart=Never --namespace test
```

```
pod/apache created
```

Ausgabe der Erzeugten Ergebnisse und die YAML Datei welche den Pod beschreibt:

<http://dukmaster-10-default.mshome.net:32188/notebooks/work/09-1-kubectl.ipynb>

Dashboard: <https://dukmaster-10-default.mshome.net:30443/>



Übung 09-2: Basis Ressourcen

Übung: kubectl-CLI und Basis Ressourcen (YAML Variante)

Das `kubectl`-Kommando stellt, eine der Schaltzentralen des K8s Clusters zur Administration der Ressourcen dar.

Die `YAML` Beschreiben die Ressourcen und Vereinfachen so die Verwendung des `kubectl` Kommandos.

In dieser Übung verwenden wir das `kubectl`-Kommando mit `YAML` Dateien zur Erstellen eines Pods und Services.

Das passiert in einer eigenen Namespace um die Resultate gezielt Darstellen zu können:

```
! kubectl create namespace yaml  
  
namespace/yaml created
```

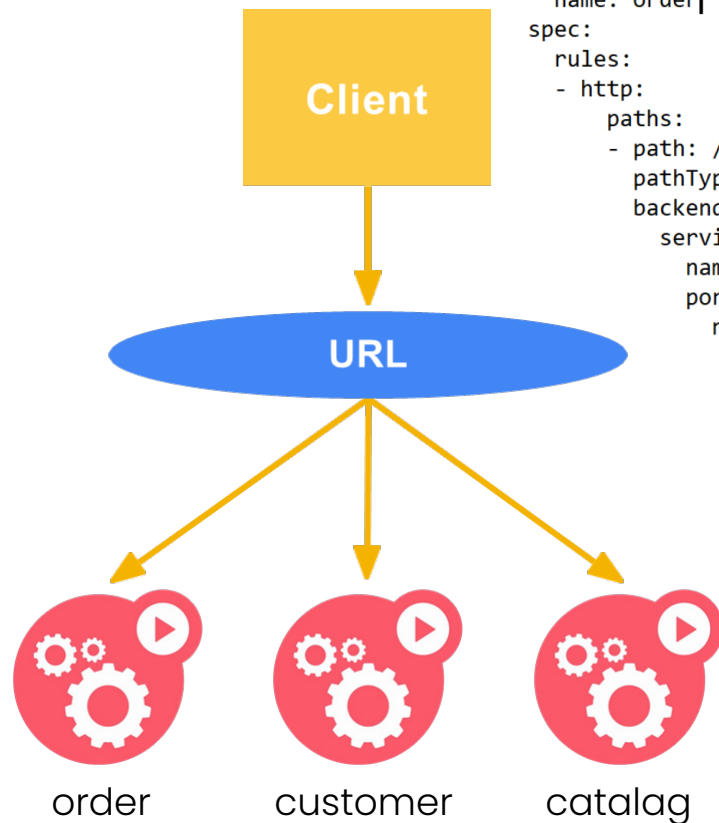
Für den Pod haben wir, die Ausgabe von `kubectl get pod apache -o yaml` genommen und eine abgespeckte Variante als YAML Datei erstellt:

```
! cat 09-2-YAML/apache-pod.yaml  
  
apiVersion: v1  
kind: Pod  
metadata:  
  labels:  
    app.kubernetes.io/name: apache  
  name: apache  
  namespace: yaml  
spec:  
  containers:  
  - image: httpd  
    name: apache
```

Ingress (Reverse Proxy, URL-based routing)

<https://<my-dns>/<Prefix>>

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: order
spec:
  rules:
  - http:
    paths:
    - path: /order/
      pathType: Prefix
      backend:
        service:
          name: order
          port:
            number: 8080
```



- Ein API-Objekt, das den externen Zugriff auf die Dienste in einem Cluster verwaltet, in der Regel mittels HTTP.
- Ingress bietet Lastenausgleich, SSL termination und namenbasiertes virtuelles Hosting.
- Grob entspricht der Ingress Dienst dem Reverse Proxy Muster.
- **Beispiele (nginx):**
 - ◇ <https://<my-dns>/order/list.html>
 - ◇ <https://<my-dns>/customer>
 - ◇ <https://<my-dns>/catalog>

Ingress (K8s Dienste Ingress)

Kubernetes as a project supports and maintains [AWS](#), [GCE](#), and [nginx](#) ingress controllers.

Additional controllers

Note: This section links to third party projects that provide functionality required by Kubernetes. The Kubernetes project authors aren't responsible for these projects, which are listed alphabetically. To add a project to this list, read the [content guide](#) before submitting a change. [More information](#).

- [AKS Application Gateway Ingress Controller](#) is an ingress controller that configures the [Azure Application Gateway](#).
- [Alibaba Cloud MSE Ingress](#) is an ingress controller that configures the [Alibaba Cloud Native Gateway](#), which is also the commercial version of [Higress](#).
- [Apache APISIX ingress controller](#) is an [Apache APISIX](#)-based ingress controller.
- [Avi Kubernetes Operator](#) provides L4-L7 load-balancing using [VMware NSX Advanced Load Balancer](#).
- [BFE Ingress Controller](#) is a [BFE](#)-based ingress controller.
- [Cilium Ingress Controller](#) is an ingress controller powered by [Cilium](#).
- The [Citrix ingress controller](#) works with Citrix Application Delivery Controller.
- [Contour](#) is an [Envoy](#) based ingress controller.
- [Emissary-Ingress](#) API Gateway is an [Envoy](#)-based ingress controller.
- [EnRoute](#) is an [Envoy](#) based API gateway that can run as an ingress controller.
- [Easegress IngressController](#) is an [Easegress](#) based API gateway that can run as an ingress controller.
- F5 BIG-IP [Container Ingress Services for Kubernetes](#) lets you use an Ingress to configure F5 BIG-IP virtual servers.

- Es sind mehrere Ingress Dienste gleichzeitig möglich.

- Beispiel:

- ◇ nginx, Port 80, 443
- ◇ istio, Port z.B. 31380

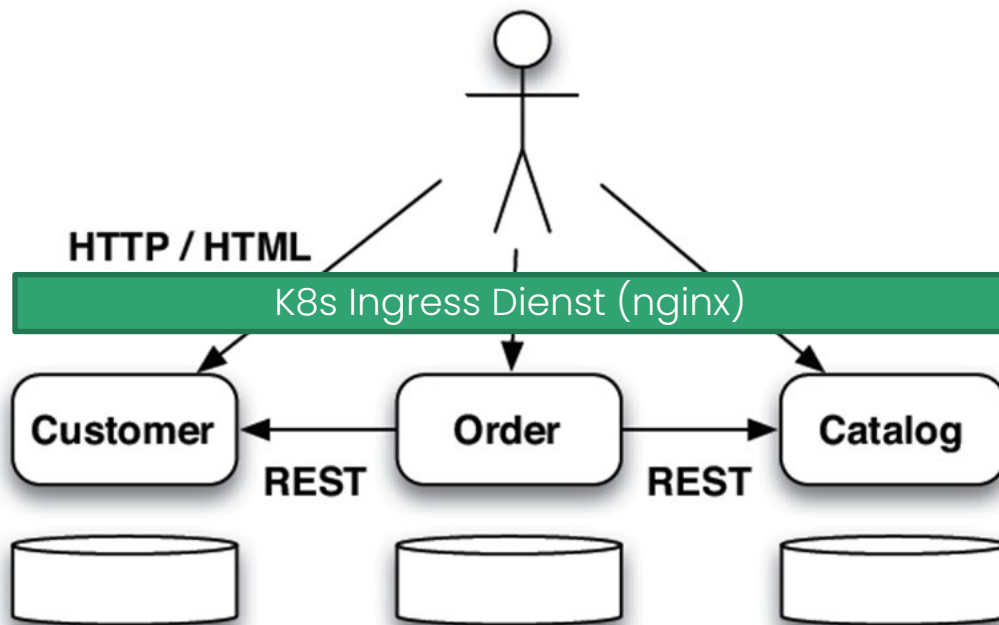


ingress-nginx	nginx-ingress-controller
istio-system	grafana-d8465c484-8wxtw
istio-system	istio-citadel-67f6594c46-
istio-system	istio-egressgateway-6ff96

- Übersicht von [Ingress Controllern](#)
- Vergleich von [Ingress Controllern](#)

Quelle: <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>

Übung 09-2: Ingress Ressource

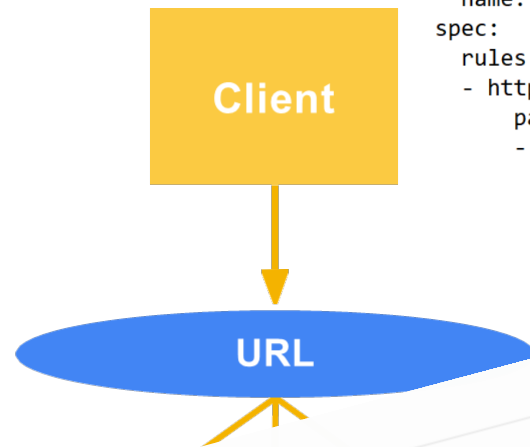


- Routing
 - ◊ Ingress
 - Service
 - ◊ Pods
 - Containers
 - Prozess(e)
- Ingress Eintrag (bzw. K8s) verändert Konfigurationsdatei vom nginx

Ingress (Reverse Proxy, URL-based routing)

<https://<my-dns>/<Prefix>>

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: order
spec:
  rules:
  - http:
    paths:
    - path: /order/
      pathType: Prefix
      backend:
        service:
          name: order
          port:
            number: 80
```



- Ein API-Objekt, das den externen Zugriff auf Dienste in einem Cluster verwaltet. Es steuert die Routing-Regel mittels HTTP

- Ingress

Note: Ingress is frozen. New features are being added to the Gateway API.

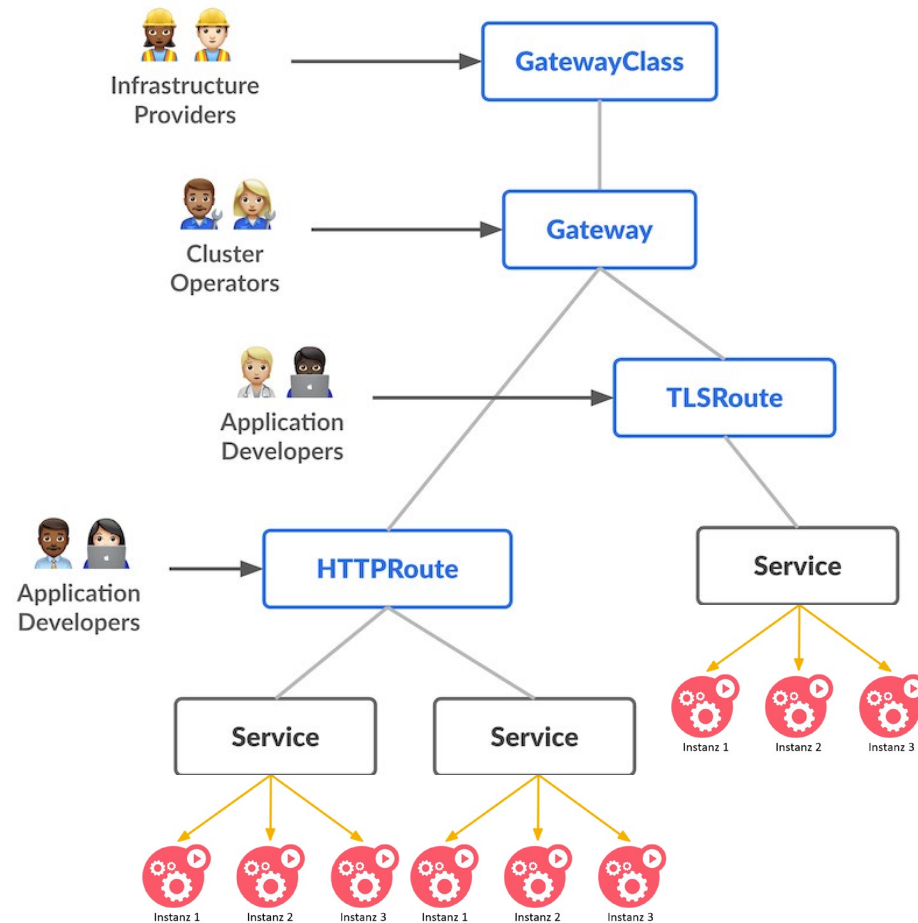
...es virtuelles

... entspricht der Ingress Dienst dem Reverse Proxy Muster.

- Beispiele (nginx):

- ◇ <https://<my-dns>/order/list.html>
- ◇ <https://<my-dns>/customer>
- ◇ <https://<my-dns>/catalog>

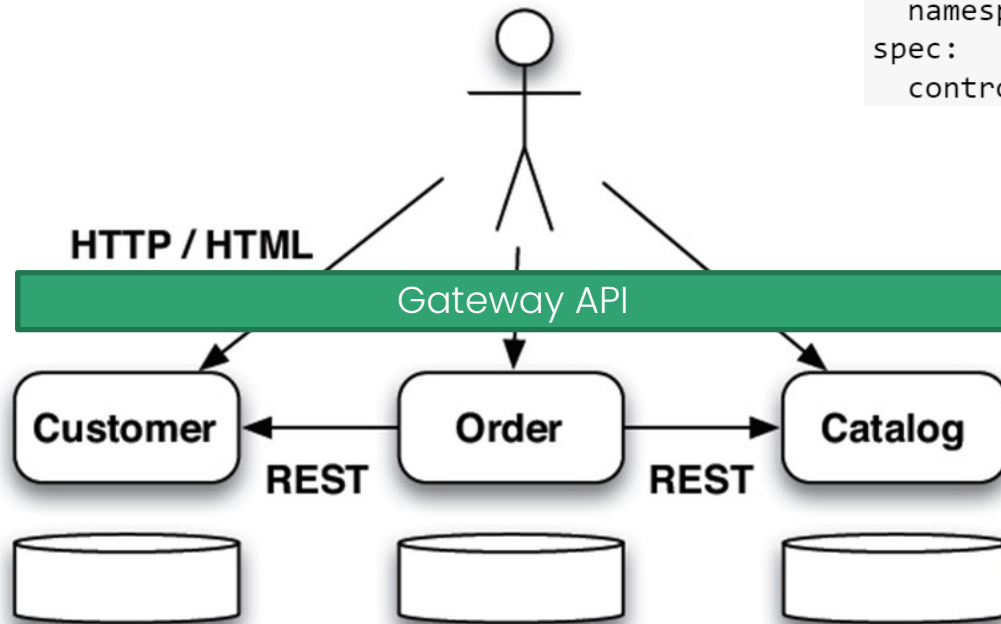
Gateway API (Reverse Proxy, URL-based routing)



- Gateway API ist ein offizielles Kubernetes-Projekt, das sich auf L4- und L7-Routing in Kubernetes konzentriert. Dieses Projekt stellt die nächste Generation von Kubernetes Ingress-, Load Balancing- und Service Mesh-APIs dar. Es war von Anfang an generisch, ausdrucksstark und rollenorientiert konzipiert.
- Das Gesamtressourcenmodell konzentriert sich auf drei separate Personas (Infrastructure, Cluster, Application) und entsprechende Ressourcen, die von ihnen verwaltet werden sollen (rechts).
- **Noch im Entwicklungsstadium!**



Übung 09-2: Gateway API



```

apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: cluster-gateway
  namespace: ms-rest
spec:
  controllerName: "ms-rest/gateway-controller"

```

```

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: ms-rest-gateway
spec:
  gatewayClassName: cluster-gateway
  listeners:
    - protocol: HTTP
      port: 80
      name: ms-rest-gw

```

```

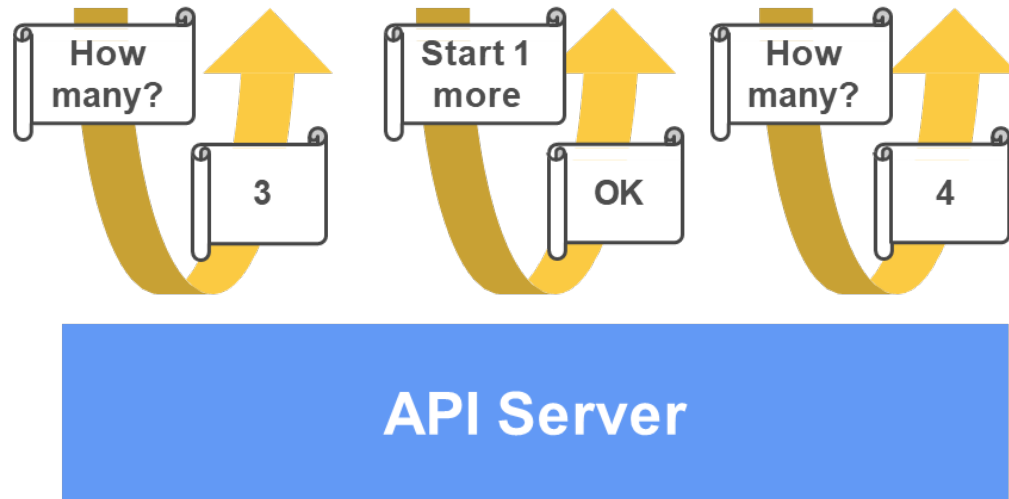
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: ms-rest-route
  namespace: ms-rest
spec:
  parentRefs:
    - name: ms-rest-gateway
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /order/(.*)
      backendRefs:
        - name: order
          port: 8080

```

Replication Controller (Neu ReplicaSets)

ReplicationController

- `selector = {"app": "my-app"}`
- `template = { ... }`
- `replicas = 4`

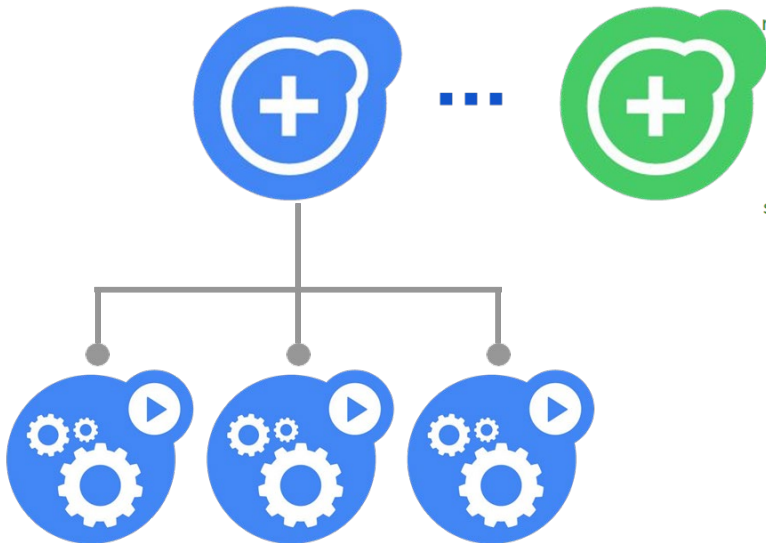


- Stellt sicher, dass N Pods (Instanzen) laufen
 - ◇ sind es zu wenig, werden neue gestartet
 - ◇ sind es zu viele werden Pods beendet
 - ◇ gruppiert durch den Label Selector
- Explizite Deklaration der gewünschten Anzahl Pods
 - ◇ ermöglicht Selbstheilung
 - ◇ erleichtert die automatische Skalierung
- Beispiel
 - ◇ Hochverfügbarer Service, z.B. Web-Shop

Deployment

Deployment

- **strategy: {type: RollingUpdate}**
- **replicas: 3**
- **selector:**
 - **app: my-app**



```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: bpmn-frontend
spec:
  replicas: 5
  selector:
    matchLabels:
      app: bpmn-frontend
  template:
    metadata:
      labels:
        app: bpmn-frontend
        group: web
        tier: frontend
    spec:
      containers:
        - name: bpmn-frontend
          image: misegr/bpmn-frontend:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
              name: bpmn-frontend

```

- Deployment eines Services (App)
 - ◇ Ersetzt ein Container Image im Pod
 - ◇ Ausgerollt durch ReplicaSet
- Ermöglicht Deklarative Updates (Deployments)
 - ◇ Erzeugt und Löscht (nach Update) automatisch ReplicaSet und Pods.

Hierarchie

- ◇ Deployment
 - ReplicaSets
 - ◇ Pods
 - Containers
 - Prozess(e)



Übung 09-3: Verteilung

Übung: Verteilung

In dieser Übung erstellen wir mehrere Pods ab dem gleichen Image mit jeweils einem ReplicaSet, Deployment und Service.

Das passiert in einer eigenen Namespace um die Resultate gezielt Darstellen zu können:

```
! kubectl create namespace rs
namespace/rs created
```

Erzeugen eines Deployments, hier der das Beispiel von Docker mit einem Web Server welche die aktuelle IP-Adresse ausgibt.

```
! kubectl run apache --image=dockercloud/hello-world --namespace rs

kubectl run --generator=deployment/apps.v1beta1 is DEPRECATED and will be removed in a future version.
deployment.apps/apache created
```

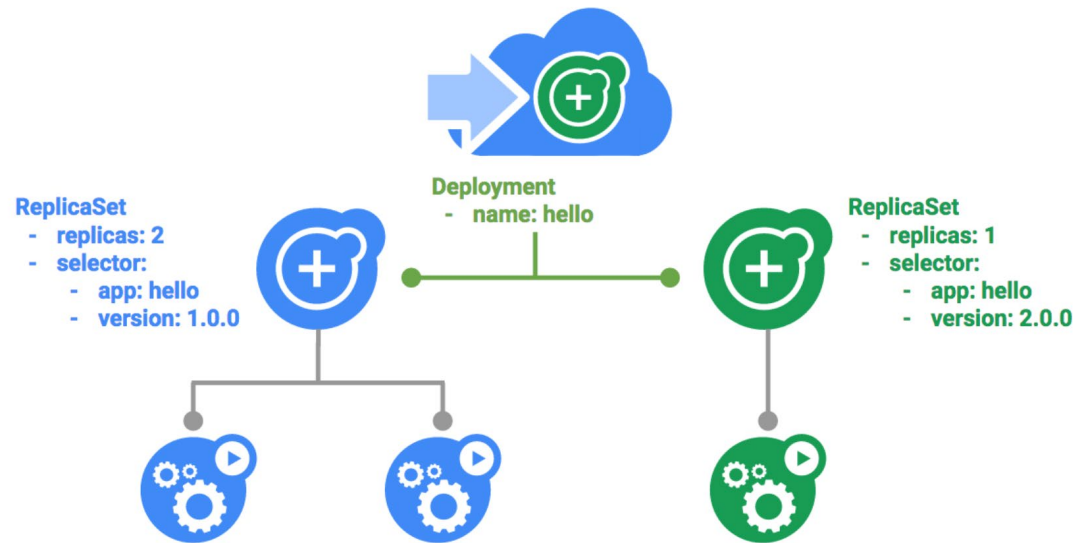
Ausgabe der Erzeugten Ergebnisse und die YAML Datei welche den Erzeugten Ressourcen beschreibt.

Ab `spec.containers` kommt erst der Pod.

```
! kubectl get pod,deployment,replicaset,service --namespace rs
```

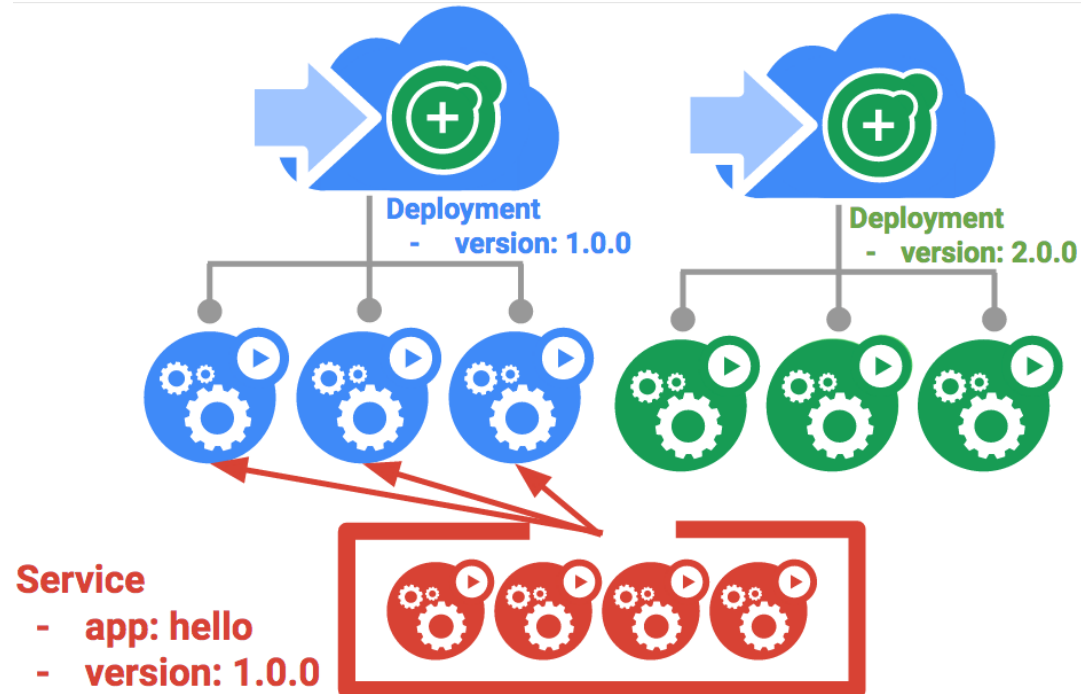
- Zusätzliche Übung:
 - ◇ Welche Microservices aus dem Beispiel [09-2-Ingress](#) sind mehrfach Auszuführen?
 - ◇ Überträgt die notwendigen Befehle von [09-3-replicaset](#) nach [09-2-Ingress](#).

Rolling Update



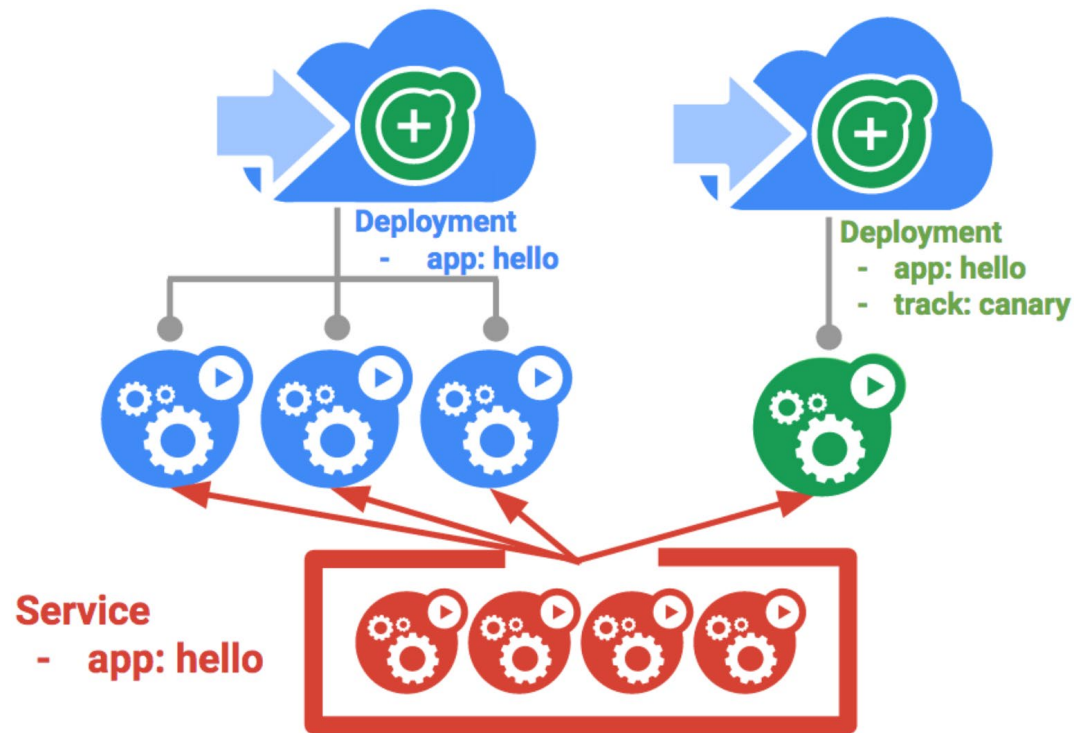
- **Deployments** unterstützen das **Aktualisieren** von **Image** auf eine neue Version mithilfe eines fortlaufenden Aktualisierungs-mechanismus.
- Wenn ein Deployment mit einer neuen Version aktualisiert wird, erstellt es ein neues ReplicaSet und erhöht kontinuierlich die Anzahl der Replikate im neuen ReplicaSet, da die Replikate im alten ReplicaSet verringert werden.

Blue/Green Deployments



- Rolling-Updates sind ideal, da sie Ihnen ermöglichen, eine Anwendung langsam mit minimalem Overhead, minimalen Auswirkungen auf die Leistung und minimalen Ausfallzeiten bereitzustellen.
- Es gibt jedoch Fälle, in denen es sinnvoll ist, die Load Balancer so zu ändern, dass sie **erst** nach der **vollständigen Verteilung** auf die **neue Version** verweisen.
- In diesem Fall sind so genannte Blue-Green-Bereitstellungen der richtige Weg.
- **Lösung:** zweites Deployment und ein Service welcher erst nach dem vollständigen Erstellen der Pods auf Version 2.0.0 umgestellt wird.

Canary Deployments



- Wenn Sie eine neue SW Version in der Produktion mit einer **kleinen Gruppe** Ihrer Benutzer testen möchten, können Sie ein «Canary» Deployment durchführen.
- Bei «**Canary**» Deployment können Sie eine Änderung nur einer **kleinen Gruppe** Ihrer Benutzer **freigeben**, um das Risiko von neuen Versionen zu verringern.
- **Lösung:** zweites Deployment, welches nur die Anzahl Canary Pods beinhaltet. Der Service steuert auf beide Deployments zu.



Übungen: Rolling Update, Blue/Green, Canary

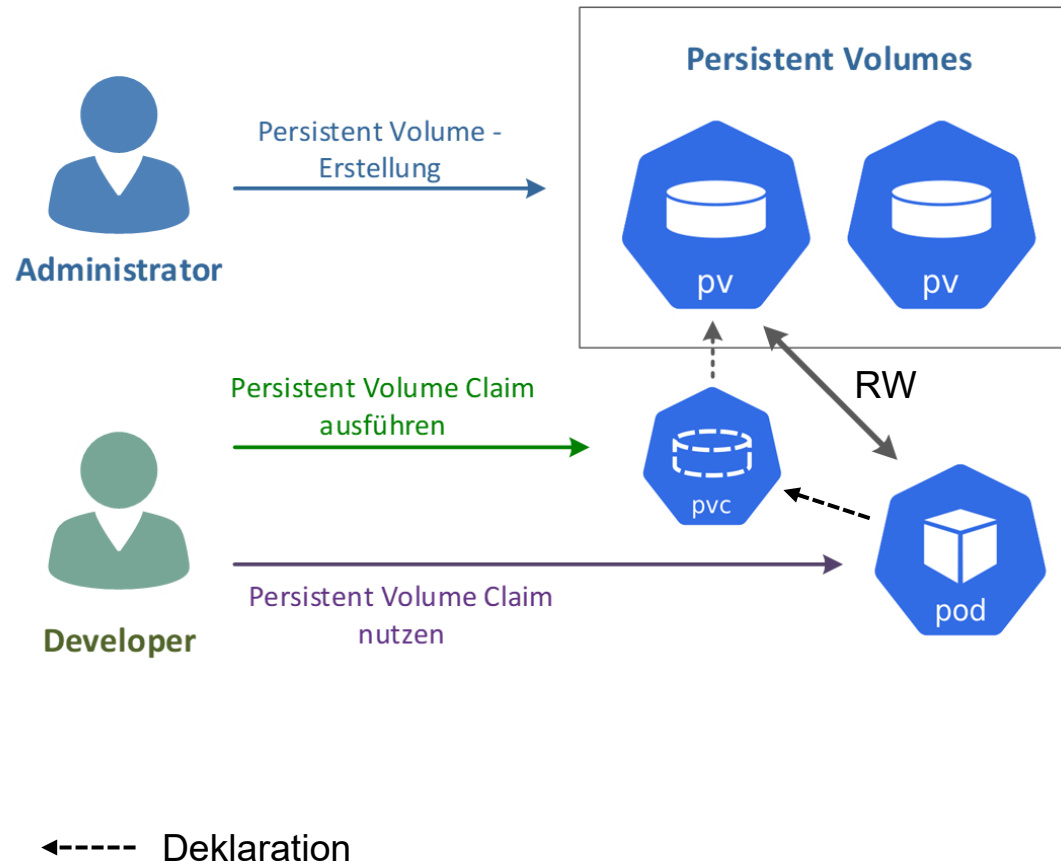
- Rolling Update
 - ◇ <http://dukmaster-10-default.mshome.net:32188/notebooks/work/09-4-Deployment.ipynb>
- Blue/Green Deployment
 - ◇ <http://dukmaster-10-default.mshome.net:32188/notebooks/work/09-4-Deployment-BlueGreen.ipynb>
- Canary Deployment
 - ◇ <http://dukmaster-10-default.mshome.net:32188/notebooks/work/09-4-Deployment-Canary.ipynb>
- Dashboard: <https://dukmaster-10-default.mshome.net:30443/>

Volumes



- Speicherplatz welcher automatisch an den Pod angehängt wird, bzw. nicht im Dateisystem des Containers gespeichert wird.
 - ◇ Cloud block storage
 - GCE Persistent Disk
 - AWS Elastic Block Storage
 - Azure File
 - ◇ Cluster storage
 - File: NFS, Ceph
 - Block: iSCSI
 - ◇ Lokale Scratch-Verzeichnisse, die bei Bedarf erstellt werden
- Kritischer Baustein für die Automatisierung.

PersistentVolume, PersistentVolumeClaims



- **PersistentVolume:** Bei PersistentVolumes (PV) handelt es sich um spezielle Storage Volume-Plugins, die Teile eines bestehenden SANs oder Storage Clusters auf vordefinierte Art an unsere Pods durchreichen können.
- Im Gegensatz zu den Volumes wird die Art, wie der Export des Volumes an den Pod erfolgt, abstrahiert.
- **PersistentVolumeClaims:** Der Pod mountet den Claim.

Hinweis

Als Storage-Ressource für ein PV muss nicht irgendein bestehendes SAN verwendet werden: Die PVs arbeiten mit den bereits vorgestellten PV-Typen wie Ceph/RBD, NFS, iSCSI, GlusterFS, GCEPersistentDisk oder auch AWSElasticBlockStore zusammen. Theoretisch auch mit hostPath, dies ist jedoch, wie bereits ausdrücklich erläutert, für Produktivumgebungen nicht empfehlenswert.

Storage Classes

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: azurefile
provisioner: kubernetes.io/azure-file
mountOptions:
  - dir_mode=0777
  - file_mode=0666
  - uid=0
  - gid=0
parameters:
  skuName: Standard_LRS
  storageAccount: k8sstore
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-claim
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
resources:
  requests:
    storage: 5Gi
```

- StorageClass bietet Administratoren die Möglichkeit, die "Klassen" des von ihnen angebotenen Speichers (grob: schnell/teuer, langsam/billig) zu beschreiben.
- Verschiedene Klassen können Servicequalitätsstufen oder Sicherheitsrichtlinien oder beliebigen Richtlinien zugeordnet werden, die von den Clusteradministratoren festgelegt werden.
- Dieses Konzept wird in anderen Speichersystemen manchmal als "Profile" bezeichnet.
- Beispiele
 - ◇ Azure: <https://github.com/mc-b/lernkube/tree/master/azure#datenspeicherung>
 - ◇ Microk8s: <https://microk8s.io/docs/addon-hostpath-storage>



Übung 09-5: Volumes und Claims

Übung: Volume und Claims

Zuerst brauchen wir ein `PersistentVolume` worauf die `Claims` zusteuern.

Im nachfolgenden Beispiel wird der `hostPath` als Volume zur Verfügung gestellt:

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: data-volume
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/data"
```

Anschließend folgen die `Claims`:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: data-claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
```

- Hierarchie
 - ◇ StorageClass
 - Persistent Volume
 - ◇ Persistent Volume Claim
- Pods / Containers
 - ◇ Zeigt auf Persistent Volume Claim
 - ◇ Legt fest: welche Verzeichnisse / Dateien werden nicht im Filesystem des Containers gespeichert

Übung 09-6: Volumes und Claims

Übung: Volume und mehrere Container in Pod

Diese Übung Demonstriert wie sich zwei Container innerhalb eines Pods das Verzeichnis `/usr/local/apache2/htdocs` teilen.

Der Container `apache` beinhaltet den Web Server und der Container `file-puller` schreibt alle 30 Sekunden die Datei `index.html` in das Verzeichnis `/usr/local/apache2/htdocs`.

Aus Einfachheitsgründen verwenden wir `emptyDir` als Volume.

Erläuterungen `emptyDir`

Das `emptyDir`-Volume wird angelegt, wenn ein Pod einem Node zugewiesen wird. Alle Container in dem Pod auf diesem (Worker-)Node können dieses `emptyDir` (einfach ein leeres Verzeichnis) lesen und schreiben.

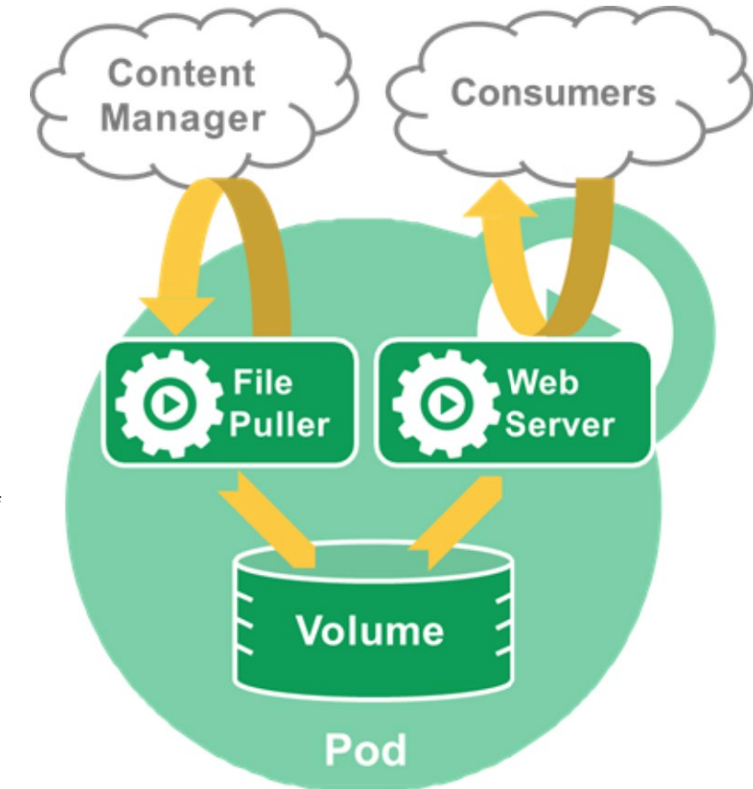
Der Pfad, mit dem das `emptyDir` innerhalb eines Containers eingehängt wird, kann sich innerhalb der Container des Pods unterscheiden.

Sobald ein Pod von einem Node gelöscht wird, wird auch der Inhalt des `emptyDir` komplett und unwiederbringlich gelöscht. Selbst wenn der gleiche Pod auf dem gleichen Worker Node neu erstellt wird, kann er nicht mehr auf das Volume seines Vorgängers zugreifen. Dies bezieht sich nicht auf den Crash eines Containers des Pods.

Typische Anwendungsfälle für `emptyDir`-Volumes könnten sein:

- Laufzeitdaten einer Applikation (Caches), die bei der Neuerstellung des Pods ohnehin neu erzeugt werden müssten
- Übergabe von (Laufzeit-)Konfigurationsdaten an alle Container innerhalb des Pods

Zuerst schauen wir den Inhalt der YAML Datei an und starten dann die Ressourcen



```
]: ! cat 09-5-Volume/web.yaml
```

<http://dukmaster-10-default.mshome.net:32188/notebooks/work/09-6-Volume.ipynb>

Dashboard: <https://dukmaster-10-default.mshome.net:30443/>

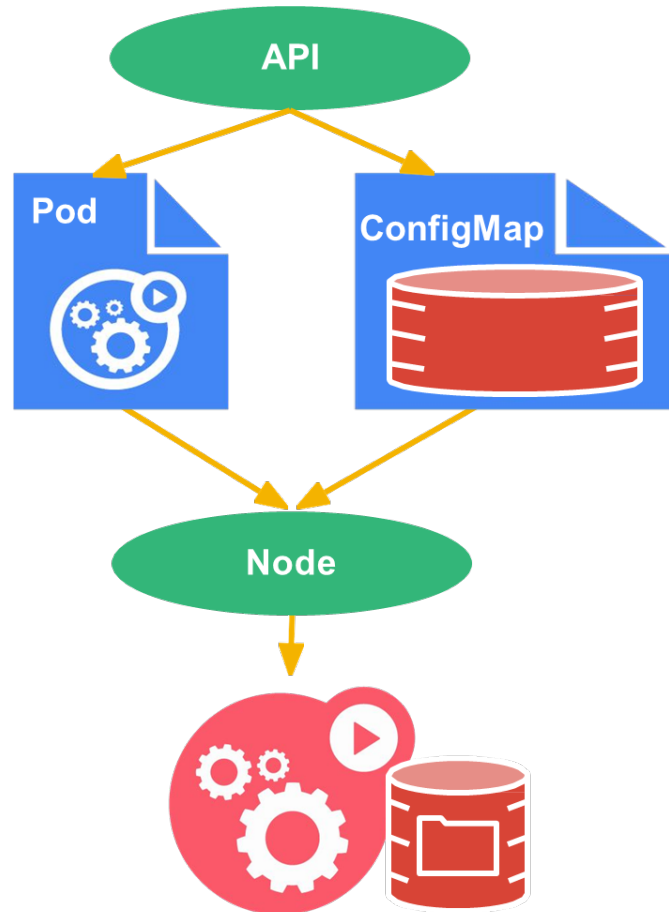


Übung: Volumes und Claims

Weitere Beispiele

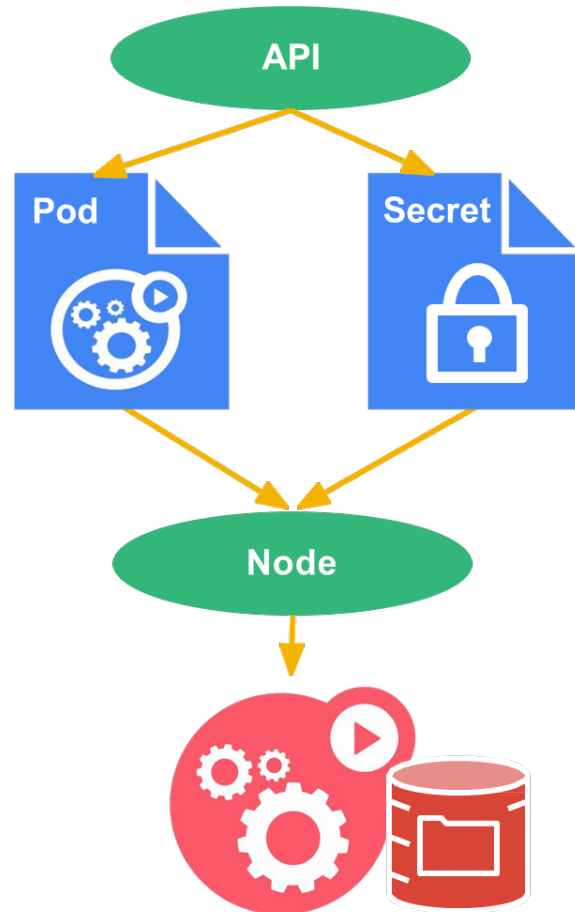
- NFS: <https://microk8s.io/docs/nfs>
- AWS: <https://github.com/mc-b/lernkube/tree/master/aws>
- Azure: <https://github.com/mc-b/lernkube/tree/master/azure>
- Docker Desktop: <https://www.docker.com/blog/docker-desktop-4-1-release-volume-management-now-included-with-docker-personal/>

ConfigMap



- ConfigMaps bieten unter K8s eine effiziente Möglichkeit, Pods auch mit komplexeren (Re-)Konfigurationen abbilden bzw. zu versorgen.
- ConfigMaps können u.a. Kommandozeilenoptionen, Umgebungsvariablen und Konfigurationsdateien sein.
- Idee ist, dass Image so standardisiert wie möglich betreiben zu können.
- D.h. Entkoppeln der Optionen, Direktiven, Umgebungsvariablen und Konfigurationsdateien vom Image.
- Über die ConfigMap-API können wir Parameter und Dateien zur Laufzeit bzw. bei der Aktivierung des Containers injizieren.
- Tutorial: <https://medium.com/google-cloud/kubernetes-configmaps-and-secrets-68d061f7ab5b>

Secrets



- Wie gewähre ich einem **Pod Zugriff** auf ein **gesichertes Objekt**?
 - ◇ secrets: credentials, tokens, passwords, etc.
 - ◇ **Speichert sie nicht im Container!**
- [12-factor](#) says should come from the environment
- Sie werden als „virtuelle Volumes“ in Pods injected, nicht in Images oder Pod-Konfigurationen eingebettet, im Speicher gehalten – sie **gelangen nie auf die Festplatte**, nicht **gekoppelt** an eine nicht-portable **Metadaten-API**.
- Geheimnisse (Secrets) werden über die Kubernetes-API verwaltet.
- [How to explain Kubernetes Secrets ...](#)
- Kubernetes [Secrets Management in 2022](#)



Übung 09-8: Configmap

Übung: ConfigMap

Am Beispiel von Apache Web Server soll der Einsatz von ConfigMap Demonstriert werden.

Zuerst müssen die ConfigMaps mittels `kubectl` erstellt werden. Vorher erstellen wir eine neue Namespace `configmap` um Erstellen Ressourcen von den anderen zu trennen:

```
In [ ]: ! kubectl create namespace configmap
! kubectl create configmap web2 --from-file=index=09-8-ConfigMap/index-2.html -n configmap
! kubectl create configmap web1 --from-file=index=09-8-ConfigMap/index-1.html -n configmap
```

Die so erstellen ConfigMaps, hätten wir auch im YAML Format beschreiben können. Deshalb geben wir uns diese im YAML Format aus:

```
In [ ]: ! kubectl get configmap -o yaml
```

Anschließend sind die Volumes in der YAML Datei so zu setzen, dass sie auf die ConfigMap Keys zugreifen

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app.kubernetes.io/name: web1
  name: web1
  namespace: configmap
spec:
  containers:
  - image: httpd
    name: apache
    volumeMounts:
    - name: config-volume
      mountPath: /usr/local/apache2/htdocs
  volumes:
  - name: config-volume
    configMap:
      name: web1
      items:
      - key: index
        path: index.html
```

<http://dukmaster-10-default.mshome.net:32188/notebooks/work/09-8-ConfigMap.ipynb>

Dashboard: <https://dukmaster-10-default.mshome.net:30443/>



Übung 09-7: Secrets (09-7-secrets)

Übung: Volume und Claims

Zuerst brauchen wir ein `PersistentVolume` worauf die `Claims` zusteuern.

Im nachfolgenden Beispiel wird der `hostPath` als Volume zur Verfügung gestellt:

```
kind: PersistentVolume
apiVersion: v1
metadata:
  name: data-volume
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/data"
```

Anschließend folgen die `Claims`:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: data-claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
```

Weitere Beispiele

- [Deploying WordPress and MySQL with Persistent Volumes](#)

Secrets – #5: Plan How You will Manage Secrets

- [Bitnami Sealed Secrets](#)
- [Godaddy Kubernetes External Secrets](#)
- [Hashicorp Vault](#)
- [Helm Secrets](#)
- [Kustomize secret generator plugins](#)



Gute Ressourcen Übersicht – <https://kubernetes.dev/>

Kubernetes Spec Explorer v1.32

Find the documentation for all Kubernetes resources, properties, types, and examples.

Select a kind from the list below to get started.

WORKLOADS

batch/v1 CronJob	apps/v1 DaemonSet	apps/v1 Deployment	batch/v1 Job
v1 Pod	apps/v1 ReplicaSet	v1 ReplicationController	apps/v1 StatefulSet

CLUSTER

v1 Event	events.k8s.io/v1 Event	v1 Namespace	v1 Node
-------------	---------------------------	-----------------	------------

NETWORKING

v1 Endpoints	discovery.k8s.io/v1 EndpointSlice	networking.k8s.io/v1 Ingress	networking.k8s.io/v1 IngressClass
networking.k8s.io/v1 NetworkPolicy	v1 Service		

CONFIGURATION

v1 ConfigMap	autoscaling/v1 HorizontalPodAutoscaler	autoscaling/v2 HorizontalPodAutoscaler	coordination.k8s.io/v1 Lease
v1 LimitRange	policy/v1 PodDisruptionBudget	v1 ResourceQuota	v1 Secret

STORAGE

storage.k8s.io/v1 CSIDriver	storage.k8s.io/v1 CSINode	storage.k8s.io/v1 CSIStorageCapacity	v1 PersistentVolume
v1 PersistentVolumeClaim	storage.k8s.io/v1 StorageClass	storage.k8s.io/v1 VolumeAttachment	

ADMINISTRATION

admissionregistration.k8s.io/v1 MutatingWebhookConfig...	scheduling.k8s.io/v1 PriorityClass	node.k8s.io/v1 RuntimeClass	admissionregistration.k8s.io/v1 ValidatingAdmissionPolicy
admissionregistration.k8s.io/v1 ValidatingAdmissionPolic...	admissionregistration.k8s.io/v1 ValidatingWebhookConfig...		

ACCESS CONTROL

rbac.authorization.k8s.io/v1 ClusterRole	rbac.authorization.k8s.io/v1 ClusterRoleBinding	rbac.authorization.k8s.io/v1 Role	rbac.authorization.k8s.io/v1 RoleBinding
v1 ServiceAccount			

OTHER

apiregistration.k8s.io/v1 APIService	certificates.k8s.io/v1 CertificateSigningRequest	certificates.k8s.io/v1alpha1 ClusterTrustBundle	v1 ComponentStatus
apps/v1 ControllerRevision	resource.k8s.io/v1beta1 DeviceClass	flowcontrol.apiserver.k8s.io/v1 FlowSchema	networking.k8s.io/v1beta1 IPAddress
coordination.k8s.io/v1alpha2 LeaseCandidate	admissionregistration.k8s.io/v1alpha1 MutatingAdmissionPolicy	admissionregistration.k8s.io/v1alpha1 MutatingAdmissionPolicy...	v1 PodTemplate
flowcontrol.apiserver.k8s.io/v1 PriorityLevelConfiguration	resource.k8s.io/v1beta1 ResourceClaim	resource.k8s.io/v1beta1 ResourceClaimTemplate	resource.k8s.io/v1beta1 ResourceSlice
networking.k8s.io/v1beta1 ServiceCIDR	internal.apiserver.k8s.io/v1alpha1 StorageVersion	storage.k8s.io/v1alpha1 StorageVersionMigration	storage.k8s.io/v1beta1 VolumeAttributesClass



Reflexion

- Kubernetes stellt eine Vielzahl von Ressourcen bereit.
- Diese werden, vorzugsweise, in YAML Dateien beschrieben.
- Die Ressourcen können mittels dem CLI `kubectl` oder über das K8s Dashboard verwaltet werden.
- Rolling Updates und deren Variationen (Canary, Blue/Green) helfen die Services auf dem neusten Stand zu halten.
- Quelle K8s Folien Core primitives und Running Microservices:
(<https://www.youtube.com/watch?v=A4A7ybtQujA>)



Lernzielkontrolle

- Sie haben einen Überblick über die Ressourcen im K8s Cluster und können dieses Einordnen.

StatefulSets

- Standard Zuordnung: Deployment, ReplicaSet, Pods (mit Container) mit UUID

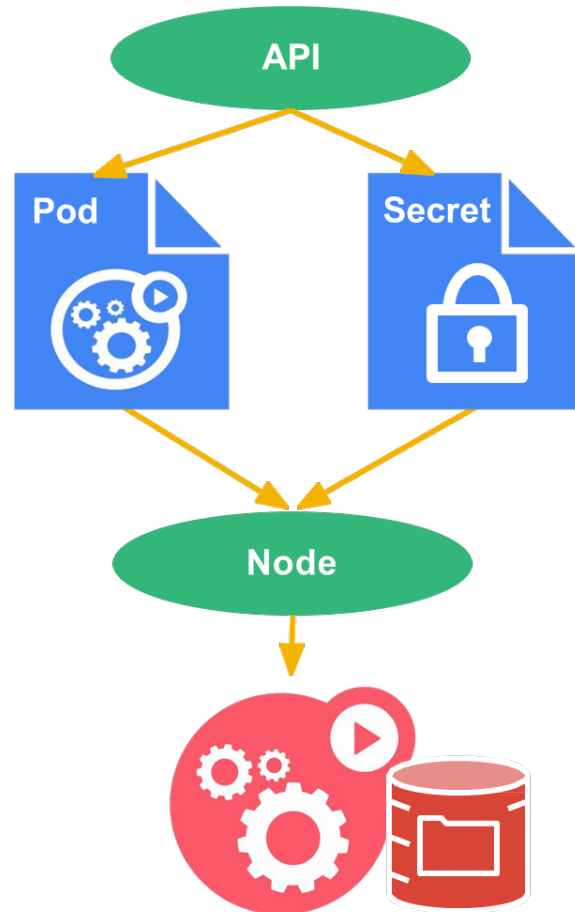
```
NAME
Deployment/bpmn-frontend
├── ReplicaSet/bpmn-frontend-5d5f785d99
│   ├── Pod/bpmn-frontend-5d5f785d99-bnx1p
│   ├── Pod/bpmn-frontend-5d5f785d99-glxs9
│   ├── Pod/bpmn-frontend-5d5f785d99-rvrqz
│   ├── Pod/bpmn-frontend-5d5f785d99-srh7t
│   └── Pod/bpmn-frontend-5d5f785d99-zzbnw
```

- StatefulSets, Pods (mit Container), mit fester Namensvergabe

```
NAME
StatefulSet/my-release-mysql
├── Pod/my-release-mysql-0
├── Pod/my-release-mysql-1
└── Pod/my-release-mysql-2
```

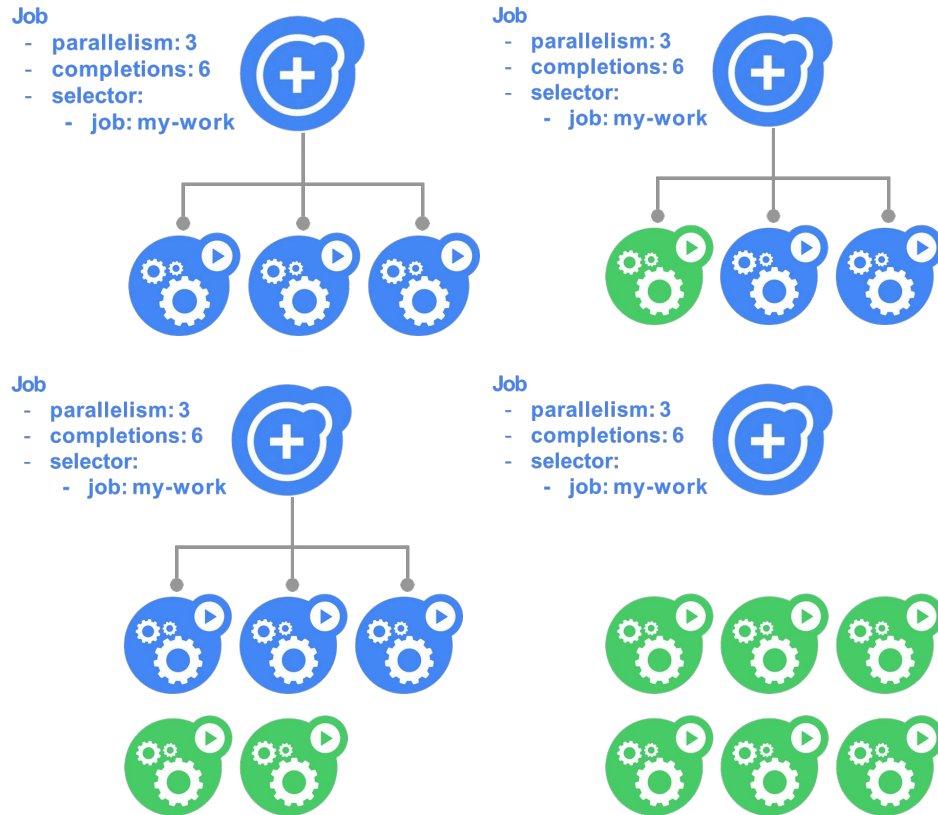
- **StatefulSets:** klassische «Long Running» Services mit persistenter, hochverfügbarer Datenablage, statischen IPs (hinter einer Service-IP) und fester Namensvergabe.
- Wichtigste Punkte:
 - ◇ persistenter Storage für die Datenablage der StatefulSets, typischerweise Cluster-/ Netzwerk-Dateisysteme, Datenbanken
 - ◇ eine feste Zuordnung im Hinblick auf IP und Namensvergabe
 - ◇ **StatefulSets müssen manuell aktualisiert werden.**
 - ◇ Zusammenspiel Persistenter Speicher und StatefulSets
<https://techblog.citystoragesystems.com/p/swapping-disks-in-kubernetes>

DaemonSet



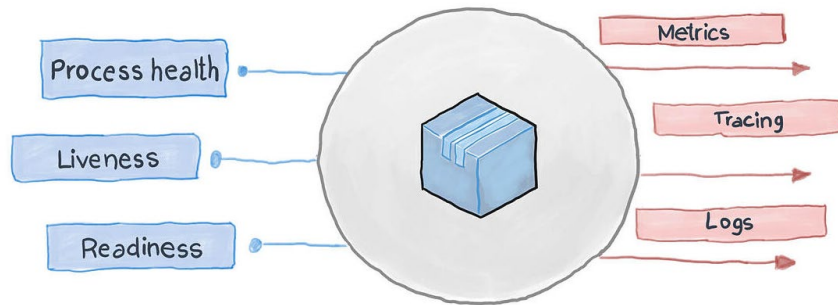
- Startet einen Pods auf jeder Node
 - ◊ oder ausgewählten Nodes
- Keine fixe Anzahl von Replicas (Instanzen)
 - ◊ erstellt und beendet durch hinzufügen oder wegnehmen von Nodes.
- Sinnvoll für Cluster weite Services wie
 - ◊ Sammeln und Auswerten von Logfiles
 - ◊ Lokale Persistenz, Caching
- Der DaemonSet Manager ist Controller und Scheduler

Jobs



- Verwaltet Pods, die als Jobs ausgeführt werden
- Ermöglicht die Anzahl der gleichzeitig ausgeführten Jobs und eine Gesamtzahl der Jobs, die durchgeführt werden müssen, festzulegen.
- Ähnlich wie ReplicationSet, aber für Pods, die nicht immer neu gestartet werden
- Workflow:
 - ◊ Neustart bei Fehler
 - ◊ Kontrolle ob sauber beendet
- Prinzip: Aufteilen und parallelisieren von Jobs, z.B. bei grossen Berechnungen.

Health Probe Pattern



- Ausfallsicherheit ist einer der wichtigsten Aspekte für geschäftskritische und hochverfügbare Anwendung.
- Eine Anwendung ist ausfallsicher, wenn sie sich schnell von Fehlern erholt.
- Das Health Probe-Pattern definiert, wie die Anwendung ihren Integritätsstatus an Kubernetes meldet.
- Dabei geht es nicht nur darum, ob der Pod betriebsbereit ist, sondern auch, ob er Anforderungen empfangen und beantworten kann.
- Mittels **Liveness**-, **Startup**- und **Readiness-Probes** erhält Kubernetes Einblick in den Gesundheitszustand des Pods und kann Entscheidungen über Verkehrsrouting und Lastausgleich treffen.

Health Probe Pattern – Umsetzung

- **Startup-Probes** (dt: Starttests) helfen, um festzustellen, wann eine Containeranwendung gestartet wurde. Wenn eine solche Tests konfiguriert ist, werden die Lebendigkeits- und Bereitschaftsprüfungen deaktiviert, bis sie erfolgreich ist, um sicherzustellen, dass diese Tests den Anwendungsstart nicht stören.
- **Readiness-Probes** (dt: Bereitschaftstests) helfen, um festzustellen, wann ein Container bereit ist, Datenverkehr anzunehmen. Ein Pod gilt als bereit, wenn alle seine Container bereit sind. Wenn ein Pod nicht bereit ist, wird er aus den Service Load Balancern entfernt.
- **Liveness-Probes** (dt: Lebendigkeitstests) helfen, um zu wissen, wann ein Container neu gestartet werden muss. Zum Beispiel könnten Liveness-Probes einen Deadlock abfangen, wenn eine Anwendung ausgeführt wird, aber keine Fortschritte erzielen kann (z.B. abhängige Datenbank startet nicht).



Übung 09-9: Health Probe Pattern (Tests.ipynb)

jupyter 09-9-Tests Last Checkpoint: vor 9 Minuten (autosaved) Python 3 C

File Edit View Insert Cell Kernel Help Not Trusted

Run Markdown

Liveness-, Readiness- und Startup-Tests

Liveness-Probes (dt: Lebendigkeitstests) helfen, um zu wissen, wann ein Container neu gestartet werden muss. Zum Beispiel könnten Liveness-Probes einen Deadlock abfangen, wenn eine Anwendung ausgeführt wird, aber keine Fortschritte erzielen kann (z.B. abhängige Datenbank startet nicht).

Das nachfolgende Beispiel startet einen Pods welcher nach 15 Sekunden abstürzt, weil die überwachte Datei nicht mehr vorhanden ist:

```
In [ ]: %%bash
cat <<%EOF% | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
    name: liveness-exec
spec:
  containers:
  - name: liveness
    image: k8s.gcr.io/busybox
    imagePullPolicy: IfNotPresent
    args:
    - /bin/sh
    - -c
    - touch /tmp/healthy; sleep 30; rm -rf /tmp/healthy; sleep 600
    livenessProbe:
      exec:
        command:
        - cat
        - /tmp/healthy
      initialDelaySeconds: 5
      periodSeconds: 5
%EOF%
```

Init Container

- Init Container sind Spezialcontainer, die vor App-Containern in einem Pod ausgeführt werden.
- Init-Container können Dienstprogramme oder Setup-Skripte enthalten, die in einem App-Image nicht vorhanden sind.
- Init-Container sind genau wie normale Container, ausser:
 - ◊ Init-Container werden immer vollständig ausgeführt.
 - ◊ Jeder Init-Container muss erfolgreich abgeschlossen sein, bevor der nächste gestartet wird.
- Wenn der Init-Container eines Pods nicht sauber beendet wird, startet Kubernetes den Pod neu, bis der Init-Container erfolgreich ist.

Init Container – Beispiele

- Warten auf einen Dienst:
 - ◇

```
for i in {1..100}; do sleep 1;  
    if dig myservice; then exit 0; fi;  
done; exit 1
```
- Registrieren des Pod bei einem Remote-Server:
 - ◇

```
curl -X POST  
http://$MNGT_SERVICE_HOST:$MNGT_SERVICE_PORT/register  
-d 'instance=$( <POD_NAME> ) &ip=$( <POD_IP> ) '
```
- Einige Zeit warten, bevor Sie den App-Container starten:
 - ◇

```
sleep 60
```



Übung 09–10: Init Container ([Init.ipynb](#))

Jupyter 09-10-Init Last Checkpoint: vor ein paar Sekunden (autosaved)

File Edit View Insert Cell Kernel Help

Init Container

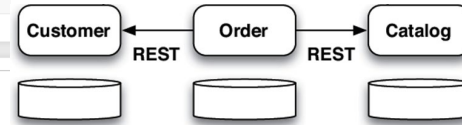
Init Container sind Spezialcontainer, die vor App-Containern in einem Pod ausgeführt werden.

Das nachfolgende Beispiel definiert einen einfachen Pod mit zwei Init-Containern.

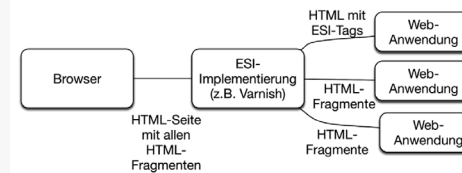
Der erste wartet auf `myservice` und der zweite wartet auf `mydb`. Sobald beide Init-Container abgeschlossen sind, wird der `spec` Abschnitt aus.

```
In [ ]: %bash
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
    - name: myapp-container
      image: busybox:1.28
      command: ['sh', '-c', 'echo The app is running! && sleep 3600']
  initContainers:
    - name: init-myservice
      image: busybox:1.28
      command: ['sh', '-c', 'until nslookup myservice; do echo waiting for myservice; sleep 2; done']
    - name: init-mydb
      image: busybox:1.28
      command: ['sh', '-c', 'until nslookup mydb; do echo waiting for mydb; sleep 2; done']
EOF
```

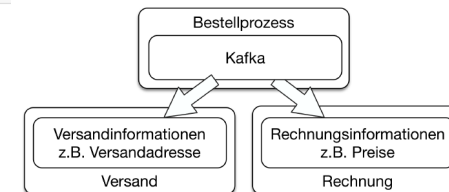
Synchrone Microservices



Frontend Integration

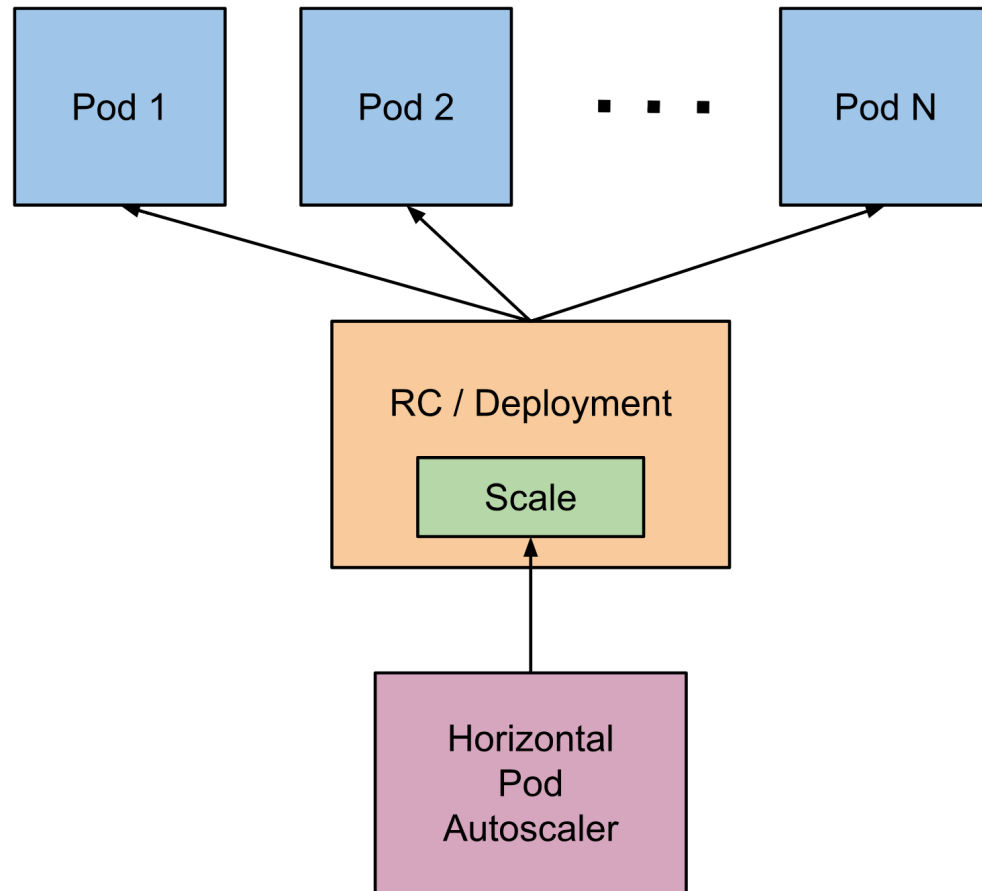


Asynchrone Microservices



- Zusätzliche Übung:
 - ◇ Für welche Microservice Konzepte bieten sich Init Container an und warum?

Horizontal Pod Autoscaler

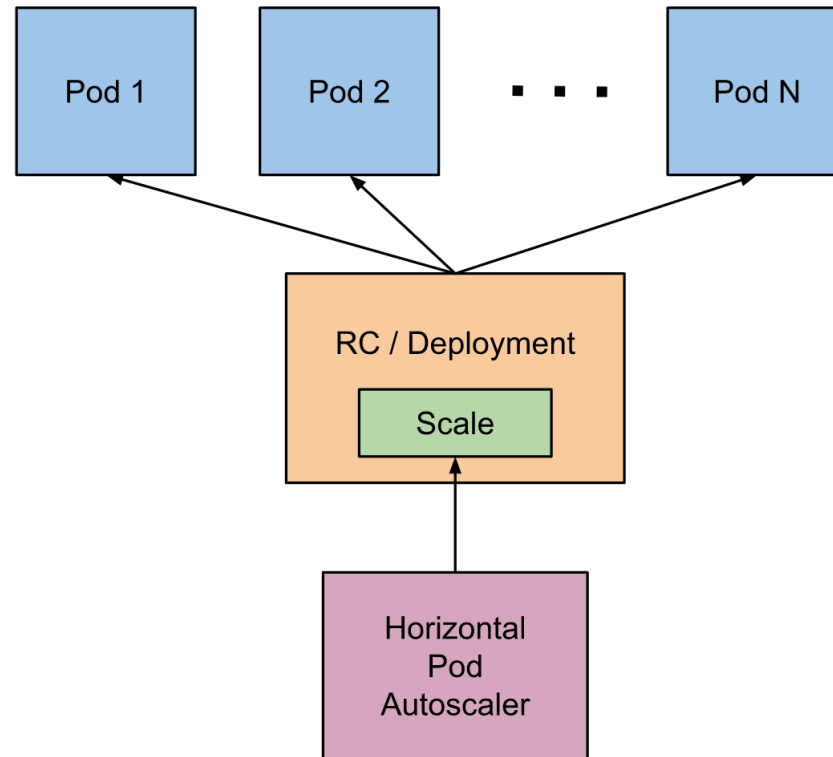


- Bei der horizontalen Pod-Autoskalierung skaliert Kubernetes entsprechend den vorgegebenen **Regelwerken vollautomatisch** die **Anzahl der Pods** in einem Replication Controller, Deployment oder ReplicaSet.
- Dies kann beispielsweise auf Grundlage der überwachten CPU-Auslastung der zugehörigen Pods erfolgen.
- Tutorial: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>



Übung 09-8: Horizontal Pod Autoscaler (HPA)

Übung Horizontal Pod Autoscaler



Container Storage Interface (CSI) ab V1.13!



- CSI Kubernetes ist ein leistungsstarkes Volume-Plugin-System um Unterstützung für neue Volume-Plugins hinzuzufügen:
- Volume-Plugins waren "in-tree", d.h. ihr Code war Teil des Kern-Kubernetes-Codes und wurde mit den Kern-Kubernetes-Binärdateien ausgeliefert.
- Weitere Informationen:
- <https://kubernetes.io/blog/2019/01/15/container-storage-interface-ga/>
- <https://kubernetes.io/docs/concepts/storage/volumes/#csi>

Persistenz: Rook a Storage Orchestrator for K8s



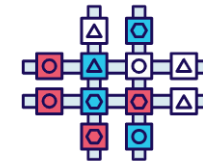
Simple and reliable automated resource management



Hyper-scale or hyper-converge your storage clusters



Efficiently distribute and replicate data to minimize loss



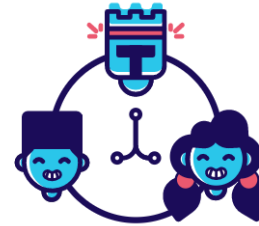
Provision, file, block, and object with multiple storage providers



Manage open-source storage technologies



Easily enable elastic storage in your datacenter



Open source software released under the Apache 2.0 license



Optimize workloads on commodity hardware